LLMOPS DEPLOYMENT GUIDE

EduAssist AI LLMOps Linux + NVIDIA RTX Deployment Guide

A step-by-step procedure for deploying the free-Ollama LLMOps stack on Debian/RedHat Linux with an NVIDIA RTX GPU and automatic CPU fallback, via a single shell script.

Birla Institute of Technology & Science

Course: MLOps (AIMLZG523) · Python 3.12 venv · Dockerised

NOVEMBER 2026

1. Overview and Prerequisites

This guide deploys the EduAssist LLMOps stack on Ubuntu/WSL or any Debian/RedHat Linux with an NVIDIA RTX GPU (CPU fallback is automatic). The entire setup is performed by a single idempotent shell script, **initial_deploy.sh**, which auto-detects the package manager, creates a Python 3.12 virtual environment, installs and starts Ollama, pulls the free model, launches MLflow, runs the evaluation pipeline, and brings up Prometheus and Grafana monitoring. Prerequisites are an NVIDIA RTX card with at least 8 GB VRAM and a working **nvidia-smi**, 16 GB RAM, roughly 6 GB of free disk for the model, sudo for package installs, and Python 3.12 (install via the Deadsnakes PPA on Ubuntu if the distro ships an older version). If any prerequisite is missing the script reports it and stops; rerun after fixing.

2. One-Shot Deployment

From the project root, make the script executable and run it. The script is safe to re-run and accepts argument options in both space and equals form. The most useful flags are summarised below.

Flag	Purpose
--model	Override the Ollama model (e.g. qwen3:8b)
--benchmark	Benchmark candidate free models and auto-pick the best
--cpu-only	Force CPU execution, ignoring the GPU
--no-monitoring	Skip the Prometheus/Grafana/API stack
--mlflow-mode local	Local sqlite MLflow instead of Docker
--skip-install	Do not apt/dnf install anything
--reset	Wipe the MLflow DB and Ollama model cache
-h, --help	Print the full help

Table 1. Selected initial_deploy.sh flags.

3. Step-by-Step Procedure

The script executes nine ordered stages. Stage 1 reads /etc/os-release and installs base packages with apt (Debian) or dnf/yum (RedHat). Stage 2 detects the GPU via nvidia-smi and sets DEVICE_TYPE. Stage 3 installs Ollama, starts the daemon, pulls the configured model, and verifies VRAM residency. Stage 4 creates the .venv with python3.12 and installs requirements. Stage 5 runs pytest and flake8 as quality gates. Stage 6 starts MLflow (Docker or local). Stage 7 warms the model with a trivial ping request to absorb the cold-load cost, then runs llmops_pipeline.py across the three prompt versions and five questions, logging all metrics to MLflow. Stage 8 optionally benchmarks candidate models and writes the winner to config/selected_model.json. Stage 9 brings up the Dockerised serving and monitoring stack and runs a smoke test plus synthetic traffic.

4. Verification and Troubleshooting

After deployment, confirm the model is resident in VRAM with **ollama ps** (the processor column should show 100% GPU) and **nvidia-smi** (roughly 4.4 GB used for qwen2.5:7b). The services are reachable at the URLs below. If **ollama ps** shows CPU in the processor column, the model is spilling to system RAM and you will reproduce the slow first-prompt symptom; switch to a smaller model with **--model gemma3:4b** or increase VRAM headroom.

Service	URL	Credentials
MLflow tracking UI	http://localhost:5000	-
FastAPI docs	http://localhost:8000/docs	-
Prometheus	http://localhost:9090	-
Grafana	http://localhost:3000	admin / admin

Table 2. Service endpoints after deployment.

Symptom	Fix
First prompt takes minutes	Check ollama ps for 100% GPU; use a smaller --model; confirm warm-up ran
nvidia-smi not found	Install the NVIDIA driver; WSL2 needs the Windows driver plus passthrough
python3.12 not found	Install via Deadsnakes PPA (Ubuntu) or pyenv
MLflow UI empty	Export MLFLOW_TRACKING_URI=http://127.0.0.1:5000 and rerun the pipeline
Port 11434 not responding	Start ollama: nohup ollama serve > logs/ollama.log 2>&1 &

Table 3. Common troubleshooting symptoms and fixes.